

Backend Engineer Intern Practical Exam

1. Assessment Overview

Objective: Develop a backend application for an ice cream shop that manages inventory items and customer records, with authentication and role-based access control.

You may choose one of the following backend technologies:

- **Option A:** Go (Golang) with Fiber
- **Option B:** Laravel with MySQL

2. Business Scenario

Imagine an ice cream shop that sells different flavors (categories), such as Chocolate, Vanilla, and Strawberry.

The shop requires a system to manage its inventory and customer records. Different users must have different levels of access depending on their roles.

The system has three user roles:

- **Owner**
- **Manager**
- **Cashier**

Your task is to design and implement a backend application that meets the following requirements.

3. Functional Requirements

3.1. Authentication and User Roles

Implement an authentication system that allows users to log in and access the application according to their assigned roles.

Requirements:

- Users must authenticate before accessing protected endpoints.
- Each user must have one of the following roles: Owner, Manager, or Cashier.
- The system must enforce role-based permissions on the backend.

You may use an appropriate authentication mechanism, such as JWT authentication for Go or Laravel Passport for Laravel. You may create the initial user accounts using database seeders or manually in the database (don’t need API endpoints).

3.2. Inventory Management

Develop CRUD APIs to manage the ice cream shop's inventory items. Searching items by name is encouraged.

3.3. Customer Management

Develop CRUD APIs to manage customer records.

4. Role-Based Access Control

The system must implement the following role-based access permissions:

1. **Owner:** Has full control over inventory items and customer records. The owner can add, view, edit, and delete both inventory items and customer records.
2. **Manager:** Can add, view, and edit inventory items and customer records. However, the manager cannot delete inventory items or customer records.
3. **Cashier:** Can add, view, and edit customer records. For inventory items, the cashier is permitted only to view and edit existing items and is not allowed to add or delete inventory items.

5. API Development Requirements

Develop RESTful APIs for authentication, inventory management, and customer management, ensuring reliability through unit tests.

The following endpoints are recommended.

Authentication

POST	/api/auth/login	Authenticate a user
POST	/api/auth/logout	Log out, if supported by the authentication method

Inventory management

GET	/api/items	Retrieve all items
GET	/api/items/{id}	Retrieve an item
POST	/api/items	Create an item
PUT/PATCH	/api/items/{id}	Update an item
DELETE	/api/items/{id}	Delete an item

Customer management

GET	/api/customers	Retrieve all customers
GET	/api/customers/{id}	Retrieve a customer
POST	/api/customers	Create a customer
PUT/PATCH	/api/customers/{id}	Update a customer
DELETE	/api/customers/{id}	Delete a customer

6. Database Design and ER Diagram

Design the database structure and create an **Entity-Relationship (ER) diagram** for the shop. Implement the database based on your proposed design.

7. Technology Requirements

Choose one of the following backend tracks.

Option A: Go Backend

Required technologies:

- Go 1.21 or higher.
- Fiber v2.
- SQLite is recommended for simplicity. Alternatively, you may use another suitable relational database.
- An appropriate database access layer (SQLC or GORM).
- Authentication and role-based authorization.

Option B: Laravel Backend

Required technologies:

- Laravel 10 or higher.
- A PHP version compatible with your selected Laravel version.
- MySQL.
- Laravel migrations and seeders.
- Appropriate authentication and authorization mechanisms.

8. Bonus: C# WPF Desktop Application

If you are interested in desktop application development, you may create a simple WPF application that consumes the APIs developed in your chosen backend track.

This is entirely optional and is not required to complete the backend assessment.

Technology requirements:

- C#
- WPF
- .NET 8 or higher

Suggested screens:

1. Login Screen

- Allow users to log in.
- Display appropriate validation and authentication errors.

2. Inventory Management Screen

- Display inventory items.
- Allow authorized users to view, add, and edit items.
- Display appropriate validation and error messages.
- Show action error messages according to the logged-in user's permissions.

3. Customer Management Screen

- Display customer records.
- Allow authorized users to view, add, and edit customers.
- Display appropriate validation and error messages.
- Show action error messages according to the logged-in user's permissions.

You may add other screens or features if you wish.

9. Submission Requirements

Push your completed project to GitHub and submit the repository link.

Your repository must contain:

- Complete source code.
- A README.md containing a project overview and setup instructions, seed data, or instructions for creating initial users and records.
- An .env.example or equivalent configuration template, where applicable.
- Instructions for running the backend.
- Instructions for running the optional WPF application, if implemented.
- Use meaningful Git commit messages and maintain a clear git history and repository structure.
- The repository should be **public**.

10. Timeline & Deliverables

- **Deadline (backend): 23rd October 2026 before 12 AM**
- **Deadline (frontend & backend): 25th October 2026 before 12 AM**

You have **3 days for the backend** and an additional **2 days for the C# app (if implemented)** to complete your work. Your work should be uploaded to a GitHub repository. Once completed, **please send your document:**

- **Including the GitHub repository**
- **Postman API collection**
- **ER diagram as a PDF, PNG, or editable source file**
- **Demo video link to Google Drive (for the C# WPF app)**

by email to the following email ID: zainab@delivergate.com **and cc**: isuru@delivergate.com, dinuka@delivergate.lk , nisal@delivergate.lk

***You must submit your final deliverables to the above-mentioned emails**. For any questions related to the assignment, email: dinuka@delivergate.lk

Good luck, and we look forward to seeing your implementation!